

La somme agentique

Interopérabilité, autonomie encadrée et fabrique de confiance : déployer des agents en services financiers réglementés (2024-2032)

André-Guy Bruneau, M.Sc. IT

Livre I — Coopérer

Fondements de l'interopérabilité et couche protocolaire : l'agent comme boucle, l'accord sur le sens que les protocoles présupposent, MCP et A2A, découverte et registres, modes d'échec.

Livre III — Encadrer

Orchestration déterministe en entreprise, cadre réglementaire canadien et terrain des services financiers : l'autonomie sous contrôle de finalité.

Livre V — Livrer et clore

L'agent comme artefact logiciel : provenance des composants, mise en service d'un artefact non reproductible, sémantique d'effet, horizon et frontière.

Livre II — Faire confiance

Identité non humaine, délégation vérifiable et fabrique de confiance : émettre un passeport opposable, tenir sous confiance hostile, compter avec l'horloge post-quantique.

Livre IV — Appliquer, exploiter, produire et composer

AgentMesh, AgentOps, fabrique d'agents et synthèse architecturale : le maillage admet, la mesure condamne, la fabrique réémet.

L'interopérabilité agentique est une propriété à maintenir, non un état atteint : elle suppose un contrat protocolaire découplé de ses mises en œuvre, une identité et une délégation vérifiables au point d'application, un contrôle de finalité opposable, une provenance attestée de l'artefact et une mesure du comportement en exploitation — cinq termes dont aucun ne supplée l'absence d'un autre.

Table des matières

Livre I Coopérer

1	L'interopérabilité comme problème d'intégration d'entreprise	2
	1.0 Introduction : l'interopérabilité comme problème d'intégration d'entreprise	2
	1.1 Fondements et théorie de l'interopérabilité	5
2	Anatomie : MCP (agent-outil) et A2A (agent-agent)	9
	8.1 MCP comme couche de contrat : problème $N \times M$, primitives, transports	9
	8.2 Jalons datés, autorisation et sémantique des résultats	13

LIVRE

I

Coopérer

*Fondements de l'interopérabilité et couche protocolaire
agentique*

L'interopérabilité comme problème d'intégration d'entreprise

Livre I — Coopérer : fondements de l'interopérabilité et couche protocolaire agentique. Premier mouvement — les fondements (ch. 1-6).

1.0 Introduction : l'interopérabilité comme problème d'intégration d'entreprise

L'interopérabilité se traite le plus souvent comme une question technique close par le choix d'un protocole ou d'un format. Le présent chapitre retient un parti pris différent, qu'il conserve jusqu'à sa clôture : la traiter comme un problème **d'intégration des systèmes d'entreprise**, c'est-à-dire comme une propriété d'ensemble que l'organisation acquiert, puis maintient, au fil de l'évolution conjointe de ses applications, de ses données et de ses processus. Ce déplacement n'est pas rhétorique. Il commande la suite de la somme : si l'interopérabilité est une propriété à maintenir, alors elle a un coût récurrent, une gouvernance, des instruments de mesure — et les protocoles agentiques des chapitres 7 à 11 en hériteront la charge sans la résoudre.

Ce chapitre est le socle **pré-agentique** du Livre I, et il est posé une seule fois. Ce qu'il établit — vocabulaire, taxonomie des niveaux, théorie du couplage, styles d'intégration, contrats d'API, messagerie, patrons de coordination — n'est reconstruit dans aucun chapitre aval : les chapitres agentiques y renvoient. Trois matières que le lecteur pourrait attendre ici en sont explicitement absentes, et leur absence est un arbitrage, non un oubli : la **sémantique** et les ontologies partent au ch. 2 ; l'**héritage IAM** — identité fédérée, autorisation déléguée, *zero trust* — part au ch. 3 ; l'**exécution durable**, les **pipelines** et l'**orchestration**

agentique partent en entier au ch. 22 (Livre III), où l'autonomie encadrée les reprend dans son propre régime.

1.0.1 Coût de la non-interopérabilité et dette d'intégration : l'enjeu économique et le périmètre du chapitre

L'absence d'interopérabilité n'est pas une gêne opérationnelle : c'est un coût continu, supporté par l'organisation qu'elle le mesure ou non. Lorsque les systèmes d'information se constituent en silos, chaque besoin d'échange impose de tisser une liaison dédiée entre deux applications. Cette intégration point-à-point engendre une croissance combinatoire du nombre de connexions : raccorder n systèmes susceptibles de dialoguer deux à deux peut requérir jusqu'à $n(n-1)/2$ liaisons, chacune avec sa logique de transport, de format et de correspondance. S'accumule alors une **dette d'intégration** : couplages rigides, transformations dupliquées, connaissances enfouies dans des adaptateurs ponctuels que plus personne n'ose modifier.

Cette dette est précisément ce que le langage des patrons d'intégration cherche à résorber, notamment par l'introduction d'un intermédiaire et d'un format pivot qui ramènent la complexité de $N \times N$ à un ordre linéaire (voir § 1.6.1.2). Le constat est donc autant économique qu'architectural : toute décision d'intégration met en balance le coût total de possession sur le cycle de vie complet — développement, exploitation, maintenance, dépréciation — et non le seul coût d'acquisition. C'est dans ce cadre que se pose l'arbitrage *build-vs-buy*, et c'est là que se loge le piège le plus fréquent : un échange réalisé à bas coût immédiat mais fortement couplé **reporte sa facture** sur l'évolution future, lorsqu'un changement unilatéral d'un côté brisera l'autre.

Le périmètre du chapitre découle de ce constat. L'angle dominant est l'intégration des systèmes d'entreprise : la capacité de systèmes hétérogènes, autonomes et distribués à coopérer. Les couches y sont parcourues du transport jusqu'à la coordination de processus. Sont délibérément exclus l'interopérabilité matérielle de bas niveau et les protocoles réseau étudiés pour eux-mêmes, retenus seulement comme socle dont les couches supérieures dépendent.

1.0.2 L'invariant transversal — découplage, contrat, évolution

Un invariant traverse la somme entière et sera rappelé à chaque couche : l'interopérabilité durable repose sur trois notions solidaires — le **découplage**, le **contrat**, l'**évolution**.

Le *découplage* est la réduction des dépendances mutuelles entre systèmes : dépendances de technologie, de format, de localisation, de temporalité et de modèle d'interaction (voir § 1.1.3). Il conditionne la mise à l'échelle, puisqu'un système faiblement couplé peut être déployé, remplacé ou modifié sans coordination synchrone avec ses partenaires. Le principe trouve sa racine dans la décomposition modulaire par masquage de l'information : un module n'expose qu'une interface stable et dissimule ses choix internes susceptibles de changer.

Le *contrat* est le support qui rend ce découplage opérationnel. C'est l'interface explicite — syntaxique, sémantique ou comportementale — sur laquelle deux parties s'accordent et qui leur permet d'évoluer indépendamment tant qu'elles la respectent (voir § 1.1.3.2). À chaque couche, le contrat prend une forme concrète : description d'API (voir § 1.4.2), schéma de message, contrat d'événement, contrat de données.

L'*évolution*, enfin, est la dimension temporelle : un contrat doit pouvoir changer sans rompre les consommateurs existants, par des règles de compatibilité ascendante et descendante, le versionnement et les lecteurs tolérants (voir § 1.1.4.1).

△ **Trois termes ici, quatre à l'avant-propos.** L'invariant de la somme compte **quatre** termes — découplage, contrat, évolution et **exploitation** —, posé en entier à l'avant-propos. Le présent chapitre en éprouve les **trois premiers** sur la matière pré-agentique ; le quatrième n'a pas d'objet ici, faute d'agent à exploiter, et se réfère au Livre IV (ch. 38-40). Invoquer le « quatrième terme » dans ce chapitre serait sans antécédent : il n'y est pas éprouvé, il y est seulement annoncé.

1.0.3 Parcours différenciés : lecture recherche et lecture praticien-architecte

La somme s'adresse à deux lectorats dont les attentes diffèrent, et le présent chapitre est conçu pour être lu selon deux parcours. Le *parcours recherche* privilégie les modèles, les taxonomies, les formalismes et les questions ouvertes ; il s'attardera sur la définition opérationnelle de l'interopérabilité, sur les modèles de maturité conceptuelle et sur les formalismes de compatibilité comportementale. Le *parcours praticien-architecte* privilégie les normes, les protocoles, les technologies et les décisions de mise en œuvre ; il suivra plus volontiers les styles d'API, les courtiers de messages et les patrons d'intégration.

Deux dispositifs, hérités du Vol. I et reconduits par la somme, servent ces deux lectures sans cloisonner le propos. Les encadrés « **Perspective recherche** »

isolent les apports théoriques — formalismes, résultats d'impossibilité — que le praticien pressé peut survoler sans perdre le fil. Les encadrés « **Mise en œuvre** » isolent à l'inverse les normes datées, les outils et les considérations de déploiement, que le lecteur recherche peut traiter comme des illustrations. Le dispositif est reconduit dans tout le Livre I ; il cesse au Livre III, dont la matière réglementaire ne s'y prête pas.

1.1 Fondements et théorie de l'interopérabilité

Avant d'aborder les cadres de référence (voir § 1.2) et les architectures d'intégration (voir § 1.3), il faut fixer un vocabulaire rigoureux. Cette section pose les définitions, les taxonomies de niveaux, la théorie du couplage et les formalismes qui justifient l'ensemble des mécanismes traités plus loin — et, au-delà, les protocoles agentiques des ch. 7 à 11, qui n'en changent pas les termes.

1.1.1 Définir l'interopérabilité : échanger et utiliser

L'interopérabilité se définit comme la capacité de deux systèmes ou plus à échanger de l'information **et à utiliser mutuellement l'information échangée**. Cette formulation distingue deux moments souvent confondus : l'échange (l'information transite) et l'usage (le destinataire interprète et exploite effectivement ce qu'il reçoit). Un transfert d'octets réussi ne garantit pas l'interopérabilité ; encore faut-il que la sémantique du message soit comprise et que l'action attendue s'ensuive. C'est cette distinction, et elle seule, qui rend intelligible l'écart que le ch. 2 nommera *accord de protocole contre compréhension* — deux agents peuvent négocier correctement un échange sans s'entendre sur ce qu'ils échangent.

Il convient également de séparer la **capacité** de l'**acte** : l'interopérabilité comme propriété d'un système (son aptitude à coopérer) ne se confond pas avec l'acte d'interopérer (une interaction effective et réussie à un instant donné). La portée est pratique : un système peut être interopérable en principe sans qu'aucun échange concret ne soit en cours, et un échange ponctuel peut réussir par accident sans propriété stable sous-jacente.

Quatre concepts voisins, fréquemment employés comme synonymes, recouvrent des relations distinctes. L'**intégration** désigne un assemblage fortement couplé où les composants sont réunis en un tout cohérent, souvent au

prix d'une dépendance mutuelle élevée. L'**interopérabilité** vise au contraire une coopération faiblement couplée : les systèmes restent autonomes et coopèrent par contrats explicites, sans fusionner. La **compatibilité** renvoie à une coexistence — deux systèmes fonctionnent côte à côte sans se nuire, sans nécessairement coopérer. La **portabilité** concerne la migration : l'aptitude d'un artefact à être transféré d'un environnement vers un autre.

Tableau 1.1 — Quatre relations entre systèmes, distinguées par le couplage qu'elles supposent.

Concept	Relation	Couplage	Finalité
Intégration	Assemblage	Fort	Constituer un tout unifié
Interopérabilité	Coopération	Faible	Échanger et utiliser l'information entre systèmes autonomes
Compatibilité	Coexistence	Nul à faible	Fonctionner sans interférence
Portabilité	Migration	—	Transférer vers un autre environnement

La distinction est structurante : l'angle dominant retenu est l'intégration des systèmes d'entreprise, mais l'objectif visé demeure l'interopérabilité au sens fort — une coopération qui préserve l'autonomie des systèmes et minimise leur couplage.

Le modèle de qualité produit SQuaRE érige enfin l'interopérabilité en sous-caractéristique **mesurable**, rattachée à la compatibilité (ISO/IEC 25010:2023), aux côtés de la coexistence. L'inscription dans un référentiel normatif a une conséquence méthodologique décisive : l'interopérabilité cesse d'être un mot d'ordre pour devenir une exigence non fonctionnelle, spécifiable, vérifiable et opposable lors de l'évaluation d'un produit logiciel. C'est ce qui permet, plus loin, de la relier au test et à la certification (ch. 3 § 3.4) plutôt qu'à une appréciation globale.

Mise en œuvre. Inscrire l'interopérabilité dans un modèle de qualité produit invite à la décliner en exigences testables dès la spécification : conformité à un contrat d'interface, tolérance aux versions, sémantique partagée. Ces exigences se transposent en contrôles automatisés dans la chaîne d'intégration continue, où elles deviennent des portes de qualité plutôt que des intentions.

1.1.2 Taxonomie des niveaux : la pile canonique et le LCIM

L'interopérabilité s'analyse en niveaux superposés formant une hiérarchie **cumulative**, où chaque palier présuppose les précédents. Le niveau **technique** assure le transport des bits et l'établissement de la connexion. Le niveau **syntactique** garantit que les structures de données échangées partagent une grammaire commune. Le niveau **sémantique** établit que les parties attribuent le même sens aux données. Le niveau **organisationnel** aligne processus métier, responsabilités et règles entre organisations. La cumulativité est essentielle : un accord sémantique est sans effet si la syntaxe diffère, et un accord syntaxique reste vain si le transport échoue.

Cette pile fournit la grille de lecture de la somme entière. Chaque mécanisme étudié plus loin se situe sur l'un de ces paliers — les formats, les schémas, la sémantique et les ontologies au ch. 2, la coordination organisationnelle des processus ici même (§ 1.6), les protocoles agentiques au ch. 8. **△ Elle prépare aussi le constat le plus important du Livre I : MCP et AzA opèrent au niveau syntaxique et, partiellement, technique ; ils ne fournissent aucun mécanisme d'accord sémantique, qu'ils présupposent.** Ce constat est établi au ch. 8 et au ch. 9 ; il est ici seulement rendu formulable.

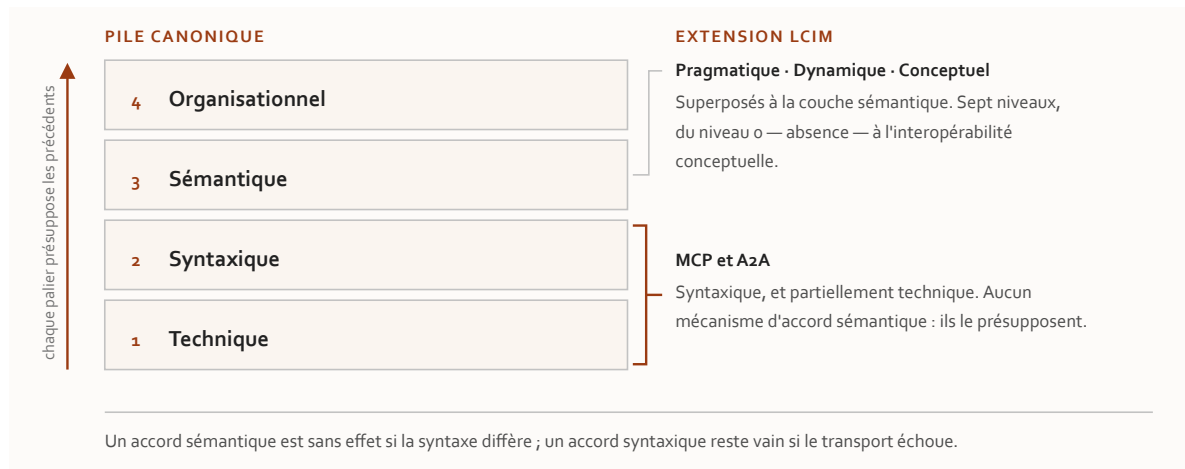


Figure 1.1 — La pile canonique d'interopérabilité, son extension par le LCIM et le palier où opèrent MCP et AzA.

Le *Levels of Conceptual Interoperability Model* (LCIM) approfondit la couche sémantique en distinguant des degrés de maturité conceptuelle de l'échange. À la pile technique-syntaxique-sémantique, il superpose les niveaux pragmatique, dynamique et conceptuel, articulés autour de trois notions : l'*integratability* (compatibilité des réseaux et plateformes), l'*interoperability* (échange et usage

des données) et la *composability* (alignement des modèles conceptuels sous-jacents). Le modèle a été raffiné en une échelle à sept niveaux, du niveau 0 — absence d'interopérabilité — jusqu'à l'interopérabilité conceptuelle, où les hypothèses et abstractions des modèles sont elles-mêmes documentées et alignées.

L'apport décisif du LCIM est de montrer qu'un échange syntaxiquement **et** sémantiquement correct ne suffit pas : tant que les modèles conceptuels et les présupposés des systèmes ne sont pas explicités, des incompréhensions subsistent, invisibles au plan technique.

Perspective recherche. Le LCIM déplace la question de l'interopérabilité du transport vers la composabilité des modèles. Cette hiérarchie conceptuelle reste un cadre d'analyse plutôt qu'un protocole opérationnel : elle qualifie ce qui manque à un échange pour être pleinement signifiant, mais ne prescrit pas comment combler l'écart — outil de diagnostic davantage qu'instrument de mesure rigoureux (limites discutées en § 1.2.2.3).

Anatomie : MCP (agent-outil) et A2A (agent-agent)

Livre I — Coopérer : fondements de l'interopérabilité et couche protocolaire agentique. Second mouvement — la couche protocolaire agentique (ch. 7-11).

△ **Trois qualifications de cette thèse sont portées par le plan et tenues ici :** la doctrine est **déclarée**, elle l'est **par une seule des deux parties**, et elle **ne contraint pas**. Chacune est instruite au § 8.6.3.

8.1 MCP comme couche de contrat : problème $N \times M$, primitives, transports

8.1.1 Le problème $N \times M$ et la critique du slogan

Ce que le protocole agent-outil standardise **n'est pas l'appel d'outil en lui-même** — la façon dont un modèle émet une invocation relève de l'ingénierie et a été traitée au ch. 4 § 4.4 — mais le **contrat** entre un client, hébergé par l'hôte agentique, et un serveur, exposé par un fournisseur de capacités.

Le problème traité est combinatoire et **familier de l'intégration d'entreprise** (ch. 1 § 1.3.1) : faire dialoguer N agents avec M outils sans standard impose $N \times M$ intégrations point-à-point ; une couche de contrat partagée ramène ce coût à $N+M$. C'est exactement le geste du modèle de données canonique (ch. 1 § 1.6.1), transposé à l'axe agent-outil — et il vaut d'être reconnu comme tel plutôt que présenté comme une nouveauté.

△ Le slogan promotionnel d'un « port universel de l'IA » capture l'ambition, mais reste un objet de critique légitime, et la critique est précise. Un port universel **négoce électriquement ses modes** ; ce protocole, lui, **ne porte pas de couche riche de négociation de capacités ni de confiance établie** entre les parties. L'appariement est conforme **au transport et au schéma, non au sens** : un client peut se brancher sur un serveur **sans qu'aucun mécanisme ne garantisse que l'intention de l'agent corresponde au comportement réel de l'outil**.

Le contrat est donc réel mais étroit : il fixe la forme de l'échange, pas son interprétation. C'est le verrou que le ch. 9 § 9.4 prend pour objet, et la première illustration concrète de la thèse du ch. 2 — les protocoles agentiques **présupposent** l'accord sémantique et ne le fournissent pas.

8.1.2 Architecture et primitives, sous l'angle de la bidirectionnalité négociée

L'architecture repose sur un triangle **hôte / client / serveur** communiquant par appels de procédure en JSON : l'hôte instancie **un client par serveur connecté**, et chaque serveur expose ses capacités.

L'apport propre à l'analyse d'interopérabilité tient moins à l'inventaire des primitives — outils, ressources, gabarits d'invite, déjà décrits au ch. 4 § 4.4 — qu'à la **grille de contrôle** qui les distingue :

Tableau 8.1 — La répartition du contrôle entre primitives : une clause du contrat, non un détail d'implémentation.

Primitive	Contrôlée par	Ce que cela prescrit
Outils	le modèle	c'est le modèle qui décide de les invoquer
Ressources	l'application	l'hôte décide quel contexte injecter
Gabarits d'invite	l'utilisateur	déclenchement explicite

△ Cette répartition est elle-même une clause du contrat : elle prescrit **qui, dans la chaîne, est autorisé à mettre une primitive en mouvement**. La lire comme un simple classement fonctionnel fait manquer ce qu'elle porte de gouvernance.

Le trait le plus distinctif au regard d'une API REST classique est l'existence de **primitives client**, qui **inversent le sens du dialogue** :

- **l'échantillonnage** — le serveur peut demander à l'hôte une complétion du modèle ;
- **la sollicitation** — le serveur peut demander une information structurée manquante, en cours d'exécution ;
- **les racines** — l'hôte délimite le périmètre de système de fichiers accessible.

Ces canaux instaurent une interopérabilité **négociée et interactive, bidirectionnelle**, absente du modèle requête-réponse unidirectionnel. *Le contrat n'est plus un appel et son retour, mais un échange où le serveur peut, à son tour, interroger l'hôte.*

☑ △ **Revalidation du 30 juillet 2026 — et c'est ici que la révision 2026-07-28 mord le plus fort.** La description ci-dessus est celle de la révision 2025-11-25, et elle reste exacte pour elle. Le texte neuf, lu à sa source, **retire le mécanisme et déprécie deux des trois primitives** : (1) les **racines** et **l'échantillonnage** passent à l'état **déprécié** — avec la **journalisation** —, la spécification recommandant de leur substituer, respectivement, le passage des répertoires par paramètres d'outil ou URI de ressource, et l'intégration directe aux interfaces des fournisseurs de modèles ; (2) le **mécanisme** de requête serveur-vers-client est remplacé par le patron des **requêtes à tours multiples** (*Multi Round-Trip Requests*, MRTR) : le serveur ne lance plus une requête à l'hôte, il **renvoie un résultat typé `input_required`** portant ses demandes, et le client **rejoue la requête d'origine** en y joignant ses réponses.

△ **Le renversement que ce paragraphe analysait n'est pas annulé — il change de forme, et la nuance est le résultat de la revalidation.** L'appelé peut toujours exiger de l'appelant une information qu'il n'avait pas ; mais il l'exige désormais **par la valeur de retour et non par un appel entrant**, ce qui rétablit un flux à sens unique au niveau du transport tout en conservant la négociation au niveau du contrat. *Le protocole de conversation subsiste ; il cesse d'être un protocole à deux appelants.* △ **La sollicitation, elle, n'est pas dépréciée** : elle passe sous MRTR, et sa notification de complétion — introduite en 2025-11-25 — est **retirée**.

Lecture de l'auteur — c'est le premier endroit de la somme où un contrat d'interface autorise explicitement l'appelé à solliciter l'appelant. Le ch. 1 § 1.1.3 posait le contrat comme publication de ce qu'un système offre et exige ; ici, il devient un **protocole de conversation**. Le socle établit l'existence des trois primitives client et la direction de leur appel ; il **n'établit ni cette lecture, ni le caractère inédit qu'elle prête à ce renversement** dans l'histoire des contrats d'interface. Elle est proposée comme telle. △ **Et la lecture survit à la revalidation sous une forme bornée** : *ce que la révision 2026-07-28 montre, c'est qu'un renversement*

de sens d'appel est une propriété coûteuse qu'un standard peut choisir de rendre à l'état de résultat typé — la bidirectionnalité était un choix, non une nécessité de la couche agentique.

8.1.3 Transports : une trajectoire du couplage vers le découplage

L'histoire des transports **se lit comme une décroissance progressive du couplage**, et c'est l'illustration la plus nette de l'invariant du Livre dans tout le mouvement.

Tableau 8.2 — La trajectoire des transports : quatre étapes, un couplage qui décroît jusqu'à l'absence d'état partagé.

Étape	Mécanisme	Couplage résiduel
Entrée-sortie standard	le serveur est un sous-proces-sus local de l'hôte	fort — cycle de vie et machine partagés
HTTP à deux points d'accès	un point pour les requêtes, un pour le flux d'événements	déprécié dès la révision suivante
HTTP diffusable à point unique	un seul point d'accès	résiduel — un identifiant de session épingle un client à une instance
Cœur sans état (<i>ratifié le 28 juill. 2026</i>)	suppression de la poignée de main, de l'identifiant de session et de la reprise de flux	aucun — serveur déployable derrière un répartiteur à tourniquet

☑ ⚠ **Revalidation du 30 juillet 2026 : la dernière étape n'est plus une cible, elle est le contrat.** Trois termes du texte neuf, lus à sa source. (1) Les **sessions de protocole** et l'en-tête qui les portait disparaissent du transport diffusable ; l'état qu'un serveur doit conserver d'un appel à l'autre passe par des **poignées explicites, frappées par le serveur et transmises comme arguments d'outil ordinaires**. (2) La **poignée de main d'initialisation** disparaît : chaque requête porte désormais sa version de protocole et les capacités du client dans ses métadonnées, et une méthode de **découverte** permet la sélection de version en amont. (3) ⚠ **Le découplage se paie, et le prix n'était pas au tableau** : la **reprise de flux et la retransmission de messages sont retirées** — un flux de réponse interrompu **perd la requête en vol**, que le client doit réémettre sous un identifiant neuf.

⚠ **C'est le fait le plus instructif de la revalidation, et il corrige la lecture d'origine.** Le tableau lisait la trajectoire comme une décroissance monotone du couplage ; la révision montre que *la dernière marche échange un couplage*

d'infrastructure contre une obligation de réémission côté client. Le couplage ne disparaît pas davantage ici qu'ailleurs : il se déplace vers l'appelant — exactement ce que le ch. 1 § 1.3 énonce, et que ce chapitre croyait pour une fois pouvoir écarter. L'**HTTP à deux points d'accès**, déprécié depuis 2025-03-26, est en outre **reclassé** sous la politique de cycle de vie neuve.

8.1.4 Une interface d'outillage assortie d'un cadre d'autorisation

△ **Formulation imposée, tenue ici et à ses trois autres occurrences marquées (réserve F-01 du Vol. II) : ce protocole est assorti d'un cadre d'autorisation, jamais d'un protocole « sécurisé ».** La distinction n'est pas de style. Un cadre fournit les mécanismes ; **la sécurité dépend de l'implémentation qui les met en œuvre**, et le ch. 11 expose ce que le socle nomme comme risques attachés. Écrire « protocole sécurisé » attribuerait à la spécification une propriété que seule une mise en œuvre peut porter — et c'est exactement ce que le garde-fou R-02 du Vol. III proscriit en matière cryptographique.

8.2 Jalons datés, autorisation et sémantique des résultats

8.2.1 Les cinq jalons : trajectoire de maturation d'un standard

La succession des révisions constitue une **étude de cas de gouvernance d'un standard** (ch. 1 § 1.2.2), où chaque jalon ajoute une couche au contrat.

Tableau 8.3 — Cinq jalons en moins de deux ans, et un cinquième qui n'était pas acquis au gel — il l'est depuis.

Révision	Date	Apports majeurs pour le contrat
2024-11-05	5 nov. 2024	version initiale ; primitives de base ; transports local et HTTP à deux points d'accès
2025-03-26	26 mars 2025	transport diffusable à point unique ; cadre d'autorisation
2025-06-18	18 juin 2025	sorties structurées ; sollicitation ; racines ; serveur qualifié serveur de ressources

Révision	Date	Apports majeurs pour le contrat
2025-11-25	25 nov. 2025	découverte d'identité ; schémas 2020-12 ; tâches asynchrones expérimentales
2026-07-28	ratifiée le 28 juill. 2026	cœur sans état — suppression des sessions de protocole et de la poignée de main ; server/discover ; cadre d'extensions ; politique de cycle de vie et de dépréciation à fenêtre de douze mois ; dépréciation des racines , de l'échantillonnage et de la journalisation ; dépréciation de l'enregistrement dynamique de client

☑ ⚠ **REVALIDATION EXÉCUTÉE LE 30 JUILLET 2026, sur la source primaire.** *L'historique de la péremption se conserve ci-dessous, au passé, parce qu'il est le seul cas du corpus où le dispositif du ch. 50 a été éprouvé par le réel (décision 17c).*

Ce que le chapitre déclarait, et ce qui a été fait. À la rédaction — 27 juillet 2026 —, le cinquième jalon était **une révision candidate et non une révision publiée** : gelée le **21 mai 2026**, sa ratification était **annoncée pour le lendemain**, et l'anatomie des § 8.1 et § 8.2 décrivait la révision **2025-11-25**. Le **28 juillet 2026**, la bascule a été **constatée** à la date annoncée (socle consolidé S-001 ; registre du volet résiduel de G-1), et le chapitre a écrit — c'était le geste juste — que *constater une bascule n'est pas lire le texte qui a basculé*, laissant la revalidation **ouverte et non exécutée**. ⚠ **Elle est exécutée ici** : le **journal des changements de la révision 2026-07-28** a été extrait à sa source le **30 juillet 2026**, et les § 8.1.2, § 8.1.3, § 8.2.1, § 8.2.2 et § 8.2.3 sont revalidés **en bloc**, comme la règle l'exigeait — *non par retouche d'un texte gelé, mais par confrontation de chaque énoncé au texte neuf*.

⚠ **Ce que la revalidation change de régime, et ce qu'elle ne change pas.** Les énoncés portant sur la révision 2026-07-28 résolvent désormais contre **un texte lu**, non contre une annonce : ils quittent le repérage pour l'extraction. ⚠ **Mais le socle n'est pas modifié par cette pièce** — S-001 et S-098 conservent leur état, et *un rédacteur ne verse pas au socle : il remonte*. Le versement est **dû**, et il est porté à la passe de socle.

△ **La divergence interne à la source, elle, n'est pas rejouée et reste consignée.** Le 28 juillet 2026, la page de versionnement déclarait encore courante la révision précédente (S-098), quand la page servie sous la révision neuve la rangeait parmi les révisions héritées **tout en la nommant courante** — divergence interne à une seule page. *Ce constat est daté du 28 juillet et ne se corrige pas depuis le 30* : il décrit l'état du site à sa date, et le registre le laisse ouvert.

(b) **Cette révision porte des changements cassants**, et non seulement des ajouts — le § 8.1.2 et le § 8.1.3 en portent désormais le détail lu. Sa ratification **a périmé l'anatomie en bloc**, et non par retouches : c'est pourquoi la revalidation a porté sur cinq sous-sections d'un coup.

(c) **La cadence elle-même est un fait d'interopérabilité** : cinq jalons en moins de deux ans témoignent d'un standard **en maturation rapide**, où la rétrocompatibilité et la dépréciation deviennent des objets de spécification à part entière. C'est un signe de maturité — et, simultanément, un coût pour qui doit épingler une version.

Perspective recherche. La densité de révisions millésimées rejoint une critique adressée à l'écosystème : *un protocole d'interopérabilité ne vaut que par la stabilité de son contrat dans le temps*, et **la profusion de spécifications versionnées par date constitue elle-même une source de fragmentation**. Une grille d'analyse sémantique situe ces gains **essentiellement aux niveaux technique et syntaxique** — transports, schémas, autorisation. Le niveau sémantique reste **repoussé vers la couche applicative** (ch. 9 § 9.4).